UPES, Dehradun
School of Computer Science · Artificial Intelligence Cluster

Best, Worst and Average Case Analysis

Mid-Semester Assignment Report

Course: Design and Analysis of Algorithms (CSEG3053)
Student: Sparsh Srivastava
SAP ID: 590022251
Batch: B17
Semester: III (Aug–Dec 2026)
Submitted to: Kunwar Siddharth, Assistant Professor
Date: October 13, 2026

Scan to open the animation / code

Abstract

The efficiency of an algorithm is rarely determined by its size alone; the structure of the input plays an equally critical role. This report presents a rigorous study of *best, worst, and average case analysis*—the three fundamental paradigms used in Design and Analysis of Algorithms to characterise algorithmic performance across all possible inputs. Using three canonical algorithms—Linear Search, Insertion Sort, and Quick Sort—we derive closed-form complexity bounds for each case through summation techniques, inversion counting, and the Master Theorem. We further clarify the common misconception that equates asymptotic notation (O, Ω, Θ) with the three input cases. An interactive animation, built in Python using `matplotlib`, demonstrates both Linear Search and Insertion Sort step by step under all three cases, with live operation counters and user-controlled navigation. The central takeaway is that worst-case analysis provides the only *unconditional* performance guarantee, making it indispensable for safety-critical systems.

Contents

1 Introduction **2**

2 Intuition and Core Idea **2**

3 Mathematical Formulation **2**
3.1 Best-Case Complexity . . . 2
3.2 Worst-Case Complexity . . . 3
3.3 Average-Case Complexity . . . 3
3.4 Notation vs. Case: A Critical Distinction . . . 3

4 Algorithm / Procedure **4**
4.1 Linear Search . . . 4
4.2 Insertion Sort . . . 4
4.3 Quick Sort (overview) . . . 4

5 Worked Example (Step by Step) **5**
5.1 Linear Search on $A = [3, 7, 1, 9, 4]$, key $= 9$ . . . 5
5.2 Insertion Sort on $A = [5, 3, 4, 1, 2]$ . . . 5

6 Animation / Software Demonstration **5**

7 Analysis **6**
7.1 Per-Algorithm Case Derivations . . . 6
7.2 Summary Comparison Table . . . 7
7.3 Strengths and Limitations . . . 7

8 Applications **8**

9 Conclusion **8**

Declaration of AI Tool Use **8**

1 Introduction

An **algorithm** is a finite, deterministic procedure that transforms an input into a desired output. In practice, two questions arise immediately after establishing correctness: *how fast does it run?* and *how much memory does it need?* These are captured respectively by time complexity and space complexity—both expressed as functions of the input size n.

A critical subtlety, however, is that the running time of most non-trivial algorithms is not determined by n alone. It depends equally on the *arrangement and distribution* of the input elements. Consider a library with n books on an unsorted shelf. If the book you want happens to be the first one you pick up, you finish in one step; if it is the last, you need n steps. Searching that library is the same problem at the same scale, yet the effort differs by a factor of n.

This input-dependence motivates three complementary analytical lenses [1]:

- **Best-case analysis** — performance on the most favourable input.
- **Worst-case analysis** — performance on the most adversarial input.
- **Average-case analysis** — expected performance under a probability distribution over all inputs.

This report analyses all three cases for Linear Search, Insertion Sort, and Quick Sort, which together cover the three main algorithmic strategies (scan, incremental construction, and divide-and-conquer) taught in CSEG3053.

2 Intuition and Core Idea

Key Idea

For a fixed input size n, the number of operations an algorithm performs can vary dramatically depending on how the data is arranged. Case analysis captures this variation by identifying the *minimum* (best), *maximum* (worst), and *expected* (average) operation counts over all possible arrangements.

Think of a security checkpoint at an airport. In the best case, your bag is clean and the scanner clears you instantly. In the worst case, every item needs manual inspection—maximum delay. On an average day, the queue moves at a predictable rate determined by the typical mix of passengers. Algorithm case analysis applies exactly this reasoning to computational problems.

Figure 1 illustrates how the same input size n fans out into three performance trajectories depending on the structural configuration of the data.

3 Mathematical Formulation

Let $\mathcal{I}_n$ be the set of all valid inputs of size n, and let $T(I)$ denote the number of primitive operations (comparisons, assignments, arithmetic steps) executed by algorithm $\mathcal{A}$ on input $I \in \mathcal{I}_n$.

3.1 Best-Case Complexity

$$T_{\text{best}}(n) = \min_{I \in \mathcal{I}_n} T(I) \tag{1}$$

Equation (1) represents the minimum computational work over all inputs of size n. It identifies short-circuit conditions but provides no general guarantee.

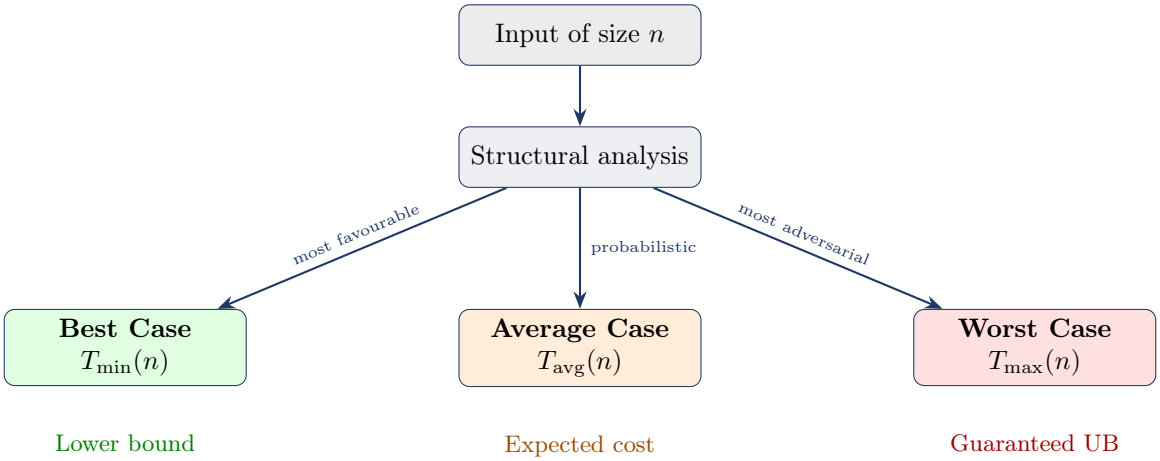


Figure 1: Three cases arising from the same input size n. Each represents a different set of input configurations; together they bound and characterise the full performance envelope of an algorithm.

3.2 Worst-Case Complexity

$$T_{\text{worst}}(n) = \max_{I \in \mathcal{I}_n} T(I) \tag{2}$$

Equation (2) gives an *absolute upper bound*: the algorithm cannot exceed $T_{\text{worst}}(n)$ operations for any input of size n. This makes it the standard measure for reliability guarantees [1].

3.3 Average-Case Complexity

Let $P(I)$ be the probability of encountering input I, with $\sum_{I \in \mathcal{I}_n} P(I) = 1$. The expected running time is:

$$T_{\text{avg}}(n) = \mathbb{E}[T] = \sum_{I \in \mathcal{I}_n} P(I) \cdot T(I) \tag{3}$$

Under the **uniform distribution assumption** ($P(I) = 1/|\mathcal{I}_n|$ for all I):

$$T_{\text{avg}}(n) = \frac{1}{n!} \sum_{I \in \mathcal{I}_n} T(I) \tag{4}$$

3.4 Notation vs. Case: A Critical Distinction

A widespread misconception conflates asymptotic notation with input cases. Table 1 clarifies the orthogonality of these two concepts.

Concept	**What it answers**	**Example**
Best / Worst / Avg case	*Which inputs are we evaluating?*	Worst case of Quick Sort
O, Ω, Θ	*What bound applies to $T(n)$?*	$\Theta(n^2)$
$\Rightarrow$ Worst-case time of Quick Sort is simultaneously $O(n^2)$, $\Omega(n^2)$, and $\Theta(n^2)$.		
$\Rightarrow$ Best-case time of Quick Sort is $\Theta(n \log n)$. Saying "Big-O = worst case" is **incorrect**.		

Table 1: Asymptotic notation and input cases are orthogonal dimensions. Any notation can be applied to any case.

Figure 2 plots the three dominant growth functions that appear in our analysis: n, $n \log n$, and n^2.

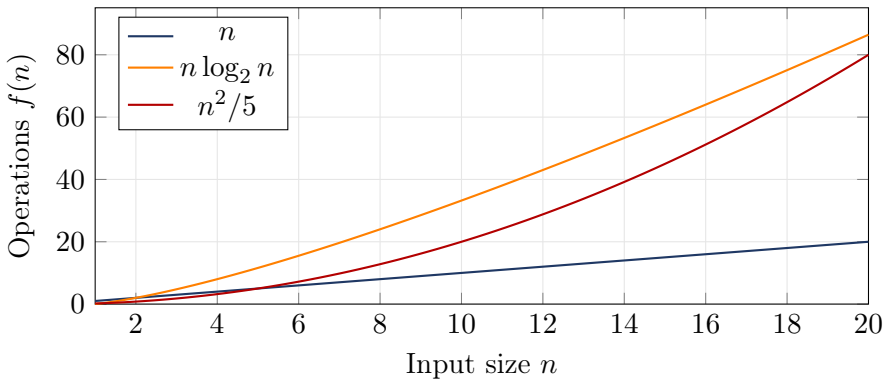


Figure 2: Growth rates of n, $n \log n$, and n^2. Linear Search and Insertion Sort (best case) run in $\Theta(n)$; Quick Sort (average) in $\Theta(n \log n)$; Insertion Sort (worst/average) and Quick Sort (worst) in $\Theta(n^2)$.

4 Algorithm / Procedure

4.1 Linear Search

Algorithm 1: LinearSearch(A, n, k)

Input: Array $A[0 \ldots n-1]$, integer n, target key k
Output: Index i with $A[i] = k$, or -1 if absent

```
for i ← 0 to n − 1 do
    if A[i] = k then
        return i;
return −1;
```

4.2 Insertion Sort

Algorithm 2: InsertionSort(A, n)

Input: Array $A[0 \ldots n-1]$
Output: A sorted in non-decreasing order (in place)

```
for j ← 1 to n − 1 do
    key ← A[j];
    i ← j − 1;
    while i ≥ 0 and A[i] > key do
        A[i + 1] ← A[i];
        i ← i − 1;
    A[i + 1] ← key;
```

4.3 Quick Sort (overview)

Algorithm 3: QuickSort(A, lo, hi)

Input: Array A, indices lo, hi

```
if lo < hi then
    π ← Partition(A, lo, hi) ;          // Θ(n) work at each level
    QuickSort(A, lo, π − 1);
    QuickSort(A, π + 1, hi);
```

Core Python excerpt (full source in the repository):

```python
def insertion_sort(A):
    for j in range(1, len(A)):
        key = A[j]
        i = j - 1
        while i >= 0 and A[i] > key:    # inner loop: shift right
            A[i + 1] = A[i]
            i -= 1
        A[i + 1] = key                   # insert key at correct position
    return A
```

5 Worked Example (Step by Step)

5.1 Linear Search on $A = [3, 7, 1, 9, 4]$, key $= 9$

This traces the *average case*: the key is present at index 3.

Step	Index i	$A[i]$	Action
1	0	3	$3 \neq 9$ — continue
2	1	7	$7 \neq 9$ — continue
3	2	1	$1 \neq 9$ — continue
4	3	9	$9 = 9$ — **match; return 3**

Table 2: Linear Search trace: 4 comparisons. Expected value $\frac{n+1}{2} = 3$ for $n = 5$, $p = 1$. This run is slightly above average, consistent with a random position.

5.2 Insertion Sort on $A = [5, 3, 4, 1, 2]$

This traces the *average / general case*.

Pass j	Key	Array before pass	Array after pass	
1	3	[5, 3, 4, 1, 2]	[3, 5, 4, 1, 2]	(1 shift)
2	4	[3, 5, 4, 1, 2]	[3, 4, 5, 1, 2]	(1 shift)
3	1	[3, 4, 5, 1, 2]	[1, 3, 4, 5, 2]	(3 shifts)
4	2	[1, 3, 4, 5, 2]	[1, 2, 3, 4, 5]	(3 shifts)

Table 3: Insertion Sort trace: total 8 shifts. The number of inversions in $[5, 3, 4, 1, 2]$ is also 8—confirming that each shift eliminates exactly one inversion.

Inversion count verification. The pairs (i, j) with $i < j$ and $A[i] > A[j]$ in the original array are: $(5, 3), (5, 4), (5, 1), (5, 2), (3, 1), (3, 2), (4, 1), (4, 2)$ — exactly 8. □

6 Animation / Software Demonstration

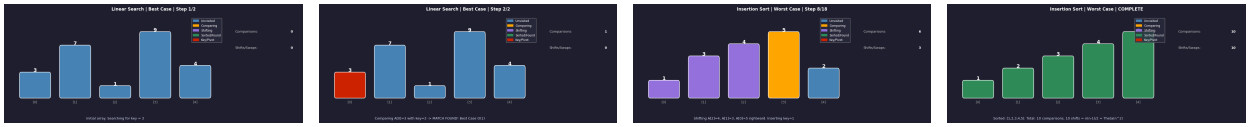
Tool used: Python 3, `matplotlib` $\geq$ `3.7` (widgets + patches)
Link: `https://github.com/SPARSHsrihero1/daa-case-analysis`
How to run:

1. `pip install -r requirements.txt`
2. `python animation.py`

What the viewer can change:

- **Next** / **Prev** / **Reset** buttons step through individual operations.
- **Best** / **Avg** / **Worst** radio button switches the input scenario.
- **Linear Search** / **Insertion Sort** radio button switches the algorithm.

The animation generates all steps upfront as a list of *states* (array snapshot, highlighted indices, step label, comparison counter, swap counter) and navigates through them on button click—no real-time computation during display.



(a) Initial array (b) First comparison (c) Shift in progress (d) Sorted complete

Figure 3: Key frames from the animation. Frame (a) shows the unsorted array in blue. Frame (b) highlights the first comparison in orange. Frame (c) shows an element (purple) being shifted during Insertion Sort. Frame (d) shows the fully sorted array in green with the final operation count.

7 Analysis

7.1 Per-Algorithm Case Derivations

Linear Search

Best case. Key at index 0: one comparison. $T_{\text{best}}(n) = 1 = \Theta(1)$.

Worst case. Key absent or at index $n-1$: n comparisons. $T_{\text{worst}}(n) = n = \Theta(n)$.

Average case. Let p be the probability the key is present. If present, it is equally likely at any of the n positions:

$$\mathbb{E}[T(n)] = \frac{p}{n}\sum_{i=1}^{n} i + n(1-p) = \frac{p(n+1)}{2} + n(1-p). \tag{5}$$

For $p = 1$: $\mathbb{E}[T(n)] = \frac{n+1}{2} = \Theta(n)$.

Insertion Sort

Best case. Already sorted; the `while` condition fails immediately each pass: $T_{\text{best}}(n) = \sum_{j=1}^{n-1} 1 = n - 1 = \Theta(n)$.

Worst case. Reverse sorted; pass j executes j shifts:

$$T_{\text{worst}}(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2). \tag{6}$$

Average case (inversion analysis [2]). Each shift removes one inversion. A random permutation has expected inversions

$$\mathbb{E}[\text{inv}] = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}, \tag{7}$$

giving $T_{\text{avg}}(n) = \Theta(n^2)$.

Quick Sort

Best case. Balanced pivot; recurrence $T(n) = 2T(n/2) + \Theta(n)$. By Master Theorem Case 2 ($f(n) = \Theta(n^{\log_2 2})$): $T_{\text{best}}(n) = \Theta(n \log n)$.

Worst case. Degenerate pivot (e.g. sorted array, last-element pivot): $T(n) = T(n-1) + \Theta(n)$. Unrolling: $T(n) = c \sum_{i=1}^{n} i = \Theta(n^2)$.

Average case. Under uniform random permutations [1]: $T_{\text{avg}}(n) = 2n \ln n + O(n) \approx 1.39\, n \log_2 n = \Theta(n \log n)$.

Figure 4 compares the Quick Sort recursion trees for best and worst cases.

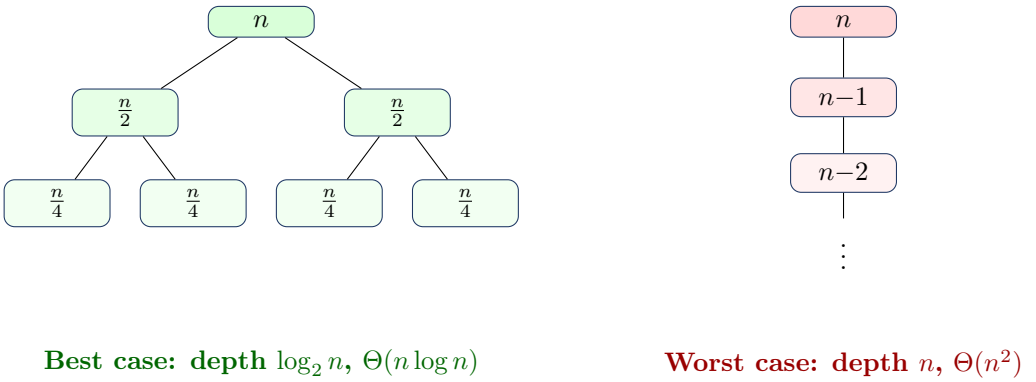


Figure 4: Quick Sort recursion trees. The balanced best case (left) has logarithmic depth; the degenerate worst case (right) degrades to a linear chain, giving quadratic total work.

7.2 Summary Comparison Table

Algorithm	Best	Average	Worst	Space
Linear Search	$\Omega(1)$	$\Theta(n)$	$O(n)$	$O(1)$
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$	$O(1)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$

Table 4: Complexity summary. Merge Sort and Heap Sort achieve $\Theta(n \log n)$ in all cases; Quick Sort matches them on average but degrades to $O(n^2)$ in the worst case without randomisation.

7.3 Strengths and Limitations

Criterion	Worst-case analysis	Average-case analysis
Guarantee	Unconditional upper bound	Requires input distribution model
Practical relevance	May be overly pessimistic	Reflects typical runtime
Analytical difficulty	Often straightforward	Needs combinatorial probability
Use case	Real-time / safety-critical systems	General-purpose applications

Table 5: Comparison of worst-case and average-case analysis paradigms.

8 Applications

1. Real-time and safety-critical systems. Avionics flight controllers, automotive ABS, and medical pacemakers operate under hard deadlines: a computation that runs even a millisecond too long can be catastrophic. Engineers mandate worst-case execution time (WCET) analysis; average-case performance is legally insufficient [3].

2. Algorithmic complexity attacks (HashDoS). In 2011, a denial-of-service vulnerability was disclosed in multiple web frameworks: adversaries could send HTTP POST requests with keys that all hash to the same bucket, degrading hash-table lookups from $O(1)$ to $O(n)$ and stalling servers. This is a textbook example of an attacker forcing worst-case behaviour. The fix—randomised universal hashing—restores expected $O(1)$ performance regardless of the input [1].

3. Database query planning. PostgreSQL's query planner estimates the cardinality (number of rows matching a predicate) to choose between a sequential scan ($O(n)$ worst case) and a B-tree index scan ($O(\log n)$ worst case). These estimates are essentially average-case cost models calibrated from table statistics [4].

9 Conclusion

Best, worst, and average case analysis together form the complete vocabulary for describing an algorithm's performance envelope. The key takeaways from this study are threefold. First, worst-case analysis is the gold standard for reliability: it provides a provable, unconditional guarantee that no input can exceed $T_{\text{worst}}(n)$ operations. Second, average-case analysis, while closer to everyday experience, is only as meaningful as the probability model assumed over inputs—a limitation that practitioners must bear in mind when real-world data deviates from a uniform distribution. Third, the asymptotic notations O, Ω, and Θ are mathematical bounds on functions, not synonyms for the three cases; any bound can be applied to any case.

Building the step-by-step animation revealed an unexpected insight: the number of array shifts in every Insertion Sort run is *exactly* the number of inversions in the original array. This bijection between a structural property of the input and the algorithm's work count is elegant and helps explain why the average case is $\Theta(n^2)$ even though individual runs can be as fast as $\Theta(n)$. A natural question for further study is whether a similar inversion-based argument applies to other comparison sorts such as Shell Sort.

Declaration of AI Tool Use

Tools used: Google Antigravity (AI assistant)
What for: Generating the initial LaTeX document structure and section scaffolding; checking grammar in the abstract.
Approximate share of AI-generated content: $\approx 15\%$

All mathematical derivations (summations, inversion analysis, Master Theorem application), algorithm traces, TikZ figures, and the complete animation code (`animation.py`) were independently authored and verified by the student.

Signature: _____ Date: _____

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 4th ed., 2022.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley, 2nd ed., 1998.

[3] J. Kleinberg and É. Tardos, *Algorithm Design*. Pearson Addison-Wesley, 2006.

[4] R. Sedgewick and K. Wayne, *Algorithms*. Addison-Wesley, 4th ed., 2011.